

\$RMHT — R Milestone HODL Token

Proof-of-Performance token on Robinhood Chain: half the supply locked in a Smart-Vault, released to holders milestone by milestone, with no admin key and no third-party keeper.

Robinhood Chain (chain 4663) · written from the source code frozen on 30 Aug 2026 · deployed 4 Sep 2026 · 322/322 tests

01 What \$RMHT is

\$RMHT is an ERC-20 token deployed on Robinhood Chain on 4 September 2026. Half of its supply — 500,000,000 tokens — is held by the token contract itself and can leave only one way: when the market cap of the token crosses a milestone and **stays above it**, the contract releases a slice of that vault to holders, pro rata.

There is no transaction tax, no team allocation beyond a time-locked founder share, no marketing cut on releases, and no privileged address able to pause, freeze, blacklist, mint or withdraw anything. After launch the configuration was locked and ownership was renounced on all four contracts. One narrow role survives, and section 9 explains exactly what it can and cannot do.

The mechanism does not depend on us. Anyone can trigger a milestone by calling a public function; nobody receives a privilege for doing so. That design choice is a direct answer to what happened to \$MHT V2 on BNB Chain, where a third-party automation service stopped and left the vault permanently unreachable.

WHAT THIS DOCUMENT IS

This whitepaper is derived from the source code frozen on 30 August 2026 and from the contracts actually deployed on 4 September 2026. Every figure quoted here is a public constant or a public variable that anyone can read from the explorer — section 13 lists the function names.

It is not investment advice, and it contains no price target, no roadmap promise and no return projection.

02 The immutable numbers

Fixed in the code at deployment. No function can change them — there is no owner left to call one.

Parameter	Value	Meaning
Total supply	1,000,000,000	Minted once, in the constructor. No mint function exists afterwards.
Smart-Vault	500,000,000	50% of supply, released milestone by milestone
Milestone step	\$500,000	Of market cap, cumulative
Release per milestone	1.5%	Of the vault balance remaining at that moment
Buy / sell tax	0%	None. Revenue comes from Uniswap LP fees.
Milestone cooldown	24 hours	Minimum delay between two releases
TWAP window	30 minutes	Full window required; the pool stores 1,801 observations
Confirmations required	3	At least 10 minutes apart, inside a 2-hour window
ETH/USD freshness limit	26 hours	A Chainlink answer older than this is rejected

03 Allocation at deployment

Allocation	Amount	Share	Where the tokens sit
Smart-Vault	500,000,000	50.00%	Held by the \$RMHT contract itself, released only through milestones
Liquidity (LP)	433,001,104	43.30%	Single-sided Uniswap v3 position, locked forever inside the LiquidityCustodian
Founder	50,000,000	5.00%	FounderCustodian, one-year time lock, immutable
Community airdrop	16,998,896	1.70%	AirdropCustodian, 18 addresses, individually claimable

About the airdrop

This is a **1-for-1** airdrop: every \$MHT token held on BNB Chain at the snapshot converts to 1 \$RMHT. It is not a pro-rata share of a fixed envelope — the 16,998,896 total is simply the sum of the real balances of the 18 retained addresses, each truncated down to a whole token.

The snapshot was frozen on 29 August 2026 and the allocation table was written into the contract at deployment in a single transaction. Nothing can be added to it afterwards.

04 The Smart-Vault and its milestones

A milestone is not crossed because the price touched a threshold once. It is crossed because market cap stayed above that threshold across several measurements spaced out in time. That is what makes an artificial pump useless here: you would have to hold the price up through thirty minutes of time-weighted average, then twenty more minutes of confirmations — something a flash loan structurally cannot do.

The four steps

- **Arming.** Market cap reaches the next milestone. The contract records the moment, counts a first confirmation, and opens a 2-hour window.
- **Confirmations.** Three confirmations are required in total, each at least 10 minutes apart. A reading taken too soon after the previous one simply does not count.
- **Disarming.** If market cap falls back below the milestone at any point, or if the two hours run out before the three confirmations, everything is cancelled and the counter restarts from zero.
- **Release.** Once the three confirmations are in and the 24-hour cooldown since the previous milestone has elapsed, 1.5% of the remaining vault is released and becomes claimable by holders. The next milestone is set \$500,000 higher.

Why 1.5% of what remains, not of the total

The calculation applies to the **remaining** vault balance, never to the original 500 million. The amount released therefore shrinks with every milestone. The vault drains asymptotically: there is no exhaustion date, and no single milestone can ever empty it. The flip side is stated in section 10 — a residual fraction stays in the contract indefinitely.

05 Who triggers a milestone

Nobody in particular. And that is deliberate.

The contract exposes a public function, `pokeMilestone()`, that anyone can call, at any time, without permission. It reads the price, compares it to the target, and advances the state machine by one step if the conditions are met. If they are not, it does nothing and costs the caller only a little gas.

There is no third-party keeper, no proprietary bot and no subscription to fund. If a single holder anywhere calls the function three times, ten minutes apart, while the price holds, the milestone fires.

WHY THIS MATTERS

On \$MHT V2, on BNB Chain, the milestone engine ran on a third-party automation service. When that dependency stopped, the vault could no longer be triggered — and ownership had already been renounced, so nothing could be repaired. Roughly 76% of that supply remains frozen to this day.

\$RMHT was rebuilt so that no external service stands between a holder and a release. A milestone cannot be forgotten, and it cannot be fired early or quietly: the function only evaluates public, verifiable conditions. Whoever calls it receives no privilege, no share and no advantage — they pay the gas, and that is all.

06 How market cap is computed

Two sources, never just one, and never an instantaneous price. Market cap is computed in two stages — the price of \$RMHT in ETH, then the price of ETH in dollars — and each stage has its own protection.

RMHT / ETH — an average, not a snapshot

The protocol never reads the pool spot price. It uses a time-weighted average over a full 30 minutes (Uniswap v3 TWAP). A single large buy therefore moves the average by only a fraction of its instantaneous effect: to actually move the number, you have to hold the position for half an hour.

The mechanism is **fail-closed**. Until the pool has accumulated a complete 30 minutes of history, no price is produced at all and `pokeMilestone()` does nothing. The protocol would rather do nothing than work from partial data. So that this window cannot be flushed by transaction spam, the pool was sized at deployment to store 1,801 observations — the full window, to the second — and the contract refuses to be configured against a pool that lacks that capacity.

ETH / USD — Chainlink, with a freshness check

The dollar conversion comes from the official Chainlink ETH/USD feed on Robinhood Chain. The contract rejects any reading older than 26 hours, refuses a zero or negative price, and verifies that the answering round is not older than the requested one. The 26-hour threshold is not arbitrary: the feed advertises a 24-hour heartbeat, plus two hours of jitter margin.

Three layers, not one

Manipulating a milestone means defeating all three at once: moving a 30-minute average, holding it across three confirmations spaced 10 minutes apart inside a 2-hour window, and doing so with a fresh ETH/USD price supplied by an oracle you do not control. Any single layer could be worked around. All three together are out of flash-loan territory.

07 How rewards reach holders

Pro rata to what you hold, with no registration and nothing to do to become eligible.

When a milestone fires, the released tokens are not pushed to every address one by one — that would not scale. The contract increments a global reward-per-token accumulator. Your share follows from your balance at any moment, and you collect it whenever you want by calling `claimRewards()`. Your entitlement grows as long as you hold, you lose nothing by waiting, and nothing expires.

Who counts in the calculation

Each holder share is computed against the eligible supply — total supply minus everything that must not dilute holders:

Excluded	Why
The vault (tokens still in the contract)	Nobody holds them — counting them would mean paying rewards to

Excluded	Why
	ourselves
The Uniswap pool	Liquidity is not a holder; including it would dilute everyone in favour of the pool
The burn address	Destroyed tokens, with no owner

Everything else counts, with no exceptions and no allow-list. This is also why the market cap the contract computes is much lower than the figure aggregators display: aggregators multiply the price by the **total** supply, while the contract multiplies it by the supply someone actually paid for.

08 The three custodians

Three separate contracts, with different rules, all deployed alongside the token.

Contract	What it holds	Exit rule
LiquidityCustodian	The Uniswap v3 position NFT	None. There is no function to withdraw liquidity or transfer the NFT. The position is locked permanently. The only thing that can leave is trading fees, through <code>collect()</code> — callable by anyone, and always paying the configured recipient, never the caller.
FounderCustodian	The founder 50,000,000	A one-year time lock, set at construction and immutable — no function can bring it forward. The milestone rewards those tokens accrue are, by contrast, claimable at any time.
AirdropCustodian	The airdrop 16,998,896	Claimable by each beneficiary as soon as either condition is met: one year elapsed, or the 5th milestone reached. Milestone rewards are claimable separately, without waiting for the principal to unlock.

Note the asymmetry, which is deliberate: airdrop beneficiaries can be served **before** the founder, never the other way round. The founder custodian has no early-unlock path at all.

About liquidity fees

There is no transaction tax. Revenue comes from Uniswap v3 swap fees, captured by the LiquidityCustodian. The function that collects them is open to everyone — not a weakness, but the guarantee that collection keeps working even if nobody is left to take care of it.

09 Governance and residual powers

After deployment the configuration was locked (`lockConfig()`) and ownership was renounced on all four contracts. From that point the pool is frozen forever, allocations can no longer be changed, no token can be frozen or blacklisted, and there is no pause function anywhere.

One role deliberately remains, and it deserves to be stated plainly rather than discovered in the code.

THE FEEDGOVERNOR ROLE — THE ONE RESIDUAL POWER

This role is entirely separate from ownership and **survives renouncement**. It can do exactly one thing: replace the **addresses** of the two Chainlink oracles — the ETH/USD feed and the sequencer uptime feed. It cannot touch the pool, the allocations, the vault, any balance, or any threshold or constant in the protocol. Every other administrative function is out of its reach, and that is enforced by test.

Why it exists: on \$MHT V2 an oracle dependency failed after ownership had been renounced, and there was then no way left to repair anything. `feedGovernor` is the answer to that specific failure — the minimum ability to repair a broken oracle, and nothing more.

It is held by a hardware wallet separate from the deployment key, can be transferred in two steps to a multisig, and can be permanently destroyed by its holder through `renounceFeedGovernor()` on the day the feeds are considered settled. From that day, the protocol has no residual power of any kind.

After launch	Status
Change the pool, the milestones, the supply, any threshold	Impossible — constants, or locked configuration
Add or modify an airdrop allocation	Impossible — one-shot, then ownership renounced
Freeze, blacklist or seize tokens	Impossible — the function does not exist
Pause the protocol	Impossible — the function does not exist
Withdraw liquidity	Impossible — the function does not exist in the custodian
Replace a Chainlink oracle address	Possible, by <code>feedGovernor</code> only

10 Known and accepted limitations

What is not perfect, and why this trade-off was chosen over the alternative.

The sequencer uptime check is inactive

The code knows how to verify that an L2 sequencer is running before accepting a price, and that path is tested. But Chainlink publishes no uptime feed for Robinhood Chain, and has announced it is no longer deploying them to new networks. Making the check mandatory would have blocked every price read forever, freezing half the supply. The check is therefore disabled at deployment, and `feedGovernor` can switch it on without redeployment the day a feed exists. In the meantime, the Chainlink freshness check, the 30-minute average and the three confirmations all remain active.

One airdrop allocation will most likely never be claimed

One of the 18 snapshot addresses is a custodial wallet whose holder does not control the keys. Its 150,000 tokens — 0.88% of the airdrop — were credited anyway, so that the announced rule applies with no exception in either direction. The mechanical consequence, stated here rather than hidden: that share stays counted as active and will permanently take 0.88% of the milestone rewards otherwise going to the other 17 beneficiaries.

The vault never empties completely

Because each milestone releases 1.5% of what remains, the series converges towards zero without reaching it. That is a property of the model, not a defect: it guarantees there is always something left to distribute, but it also means a residual fraction stays in the contract indefinitely.

There was a window between deployment and lock-down

The TWAP needs 30 minutes of observations before it can be used, so the final configuration lock could not run in the same breath as the deployment. There was a short period during which the token was tradable while the configuration was not yet frozen. That window ran from the deployment to the configuration lock, both on 4 September 2026; what happened inside it, and what the launch cost down to the wei, is written up on the public transparency page.

11 Deployed contracts

Deployed on Robinhood Chain (chain 4663) on 4 September 2026. The four contracts we wrote are verified on Blockscout — readable source, exact match against the deployed bytecode — and ownership has been renounced on all four.

Contract	Address	State
\$RMHT (token and vault)	0xbd4487dad62d23e0677E6a94c99cB0AE45328bA4	Verified · owner() = address(0) · configuration locked
RMHTAirdropCustodian	0xdEc68344a41Cc6104ed5ff2C88F2D16a794688C1	Verified · owner() = address(0)
RMHTFounderCustodian	0x4C944c27160A1532b9De3a69d0Ec64164a6f03d8	Verified · owner() = address(0)
RMHTLiquidityCustodian	0x50336e2a1396364895702a5222faC47bD0d38407	Verified · owner() = address(0) · position tokenId 1004464
Uniswap v3 pool (RMHT/WETH)	0xabe3B1fF5Fc6a9638D0c46f2379B20f7e6173bEe	Deployed by the Uniswap factory, not by us

The only role that survives the renounce is feedGovernor, held by **0xC90e404A9f2ea55cbD2C81cf0054FAe4A721889a**. What it can and cannot do is set out in section 9.

12 Tests, tooling and coverage

No external firm reviewed this code — section 13 says so plainly. What follows is what **was** done, with the raw numbers, so that the two halves of that sentence can be weighed against each other. Every log quoted below is archived in the public repository, under **audit/**, and every figure can be reproduced by cloning it and running the suite.

The Foundry test suite

The suite counts **322 tests** as of 30 August 2026. The version published on GitHub counts 307: two test files exercise the deployment script, which is deliberately not published. The audit campaign archived in the repository is the run of **29 August 2026** — 9 suites, **314 tests, 0 failed, 0 skipped**, in 101.9 seconds:

Test suite	Tests	What it covers
RMHT.t.sol	113	Token, vault, milestone state machine, rewards accounting, access control
RMHTFounderCustodian.t.sol	63	One-year lock, reward accrual, rescue paths, reentrancy attempts
RMHTAirdropCustodian.t.sol	56	The 18 allocations, hybrid unlock, per-beneficiary claims, batch limits
RMHTLiquidityCustodian.t.sol	35	Position custody, permissionless collect(), absence of any exit path
UniswapV3TWAP.t.sol	19	The TWAP library: tick maths, sqrt ratios, round-trips
RMHTTwap.t.sol	13	TWAP integration: fail-closed behaviour, observation cardinality
EvmVersionOsaka.t.sol	4	Behaviour under the target EVM version
RMHT.invariant.t.sol	4	Stateful invariants — see below
Total (29 Aug run)	314	0 failed, 0 skipped

Fuzzing and stateful invariants

Property tests run 256 randomised rounds each. The four stateful invariants run **256 rounds of 128,000 calls each — with 0 reverts**, meaning the sequencer never produced an input the contract refused:

- **invariant_TotalSupplyNeverChanges** — no code path can mint or destroy supply.
- **invariant_VaultBalanceNeverExceedsInitial** — the vault can only ever go down.
- **invariant_ContractHoldsAtLeastVaultBalance** — the contract always physically holds what it claims to owe the vault.
- **invariant_MilestoneCountConsistentWithVaultDrop** — a vault decrease and a milestone count can never disagree.

Over that campaign the random handler distributed its calls across **42,589 claims, 42,882 transfers and 42,529 poke-and-warp sequences**, none of which reverted. Alongside them, fuzz tests check that transfers never exceed a balance, that rewards stay strictly proportional to balances, that repeated milestones never over-draw the vault, and that the tick/sqrt-ratio conversions round-trip and stay strictly increasing.

Coverage, measured not claimed

File	Lines	Statements	Functions
src/RMHT.sol	98.42% (312/317)	95.28% (323/339)	100% (57/57)
src/RMHTAirdropCustodian.sol	99.12% (113/114)	99.24% (131/132)	100% (21/21)
src/RMHTFounderCustodian.sol	100% (82/82)	100% (92/92)	100% (20/20)
src/RMHTLiquidityCustodian.sol	100% (49/49)	100% (42/42)	100% (13/13)
src/libraries/UniswapV3TWAP.sol	100% (44/44)	100% (86/86)	100% (3/3)
Total	98.35% (836/850)	97.54% (873/895)	98.34% (178/181)

Branch coverage reads 45.54% (143/314) and is left out of the table on purpose, because quoting it without explanation would be misleading in the other direction: under **--ir-minimum**, Foundry counts branches generated by the compiler itself, which no test can reach. The line, statement and function figures are the meaningful ones.

The eight tools, and what each returned

Tool	Result on the 29 August campaign
Foundry	314/314 passing, 0 failed — the run described above
Slither	28 contracts analysed with 100 detectors, 52 results, none in the High or Critical families . The remainder are informational (naming, literal formatting, deliberate unused returns on interface probes).
Solhint	Empty log — zero warnings.
Wake	Compiled 5 files, 11 findings, all in categories already triaged and none new : 4 unsafe-erc20-call on a pattern that does check the return value, 3 unchecked-return on deliberate interface probes, 2 reentrancy flags on functions that are already nonReentrant and owner-only, 2 Low balance-relied-on.
Aderyn	2 High and 10 Low. Both Highs are known and explained: state change after an external call on functions that are already nonReentrant or owner-only setup, and contract names reused across files (IERC20, Context, ReentrancyGuard), which has no effect under Foundry.
Mythril	Symbolic execution. It took two runs — both are published. See the caveat below.
Echidna	Property-based fuzzing campaign; the configuration is published in echidna/ so it can be re-run.

Tool	Result on the 29 August campaign
SolidityScan / Surya	SolidityScan for a commercial second opinion, Surya for the call graph and inheritance graph (a documentation tool, it produces no findings).

TWO CAVEATS ON THE TOOLING, BECAUSE THEY CHANGE HOW MUCH THE RESULTS ARE WORTH

Mythril needed two runs, and both logs are published. In the 15:50 campaign it could analyse only 1 of the 5 files: its Solidity downloader points at a host that no longer resolves, so four analyses never started — a toolchain failure, with nothing to do with the contracts. A second run the same day at 16:09, after the solc 0.8.36 binary was placed where the downloader looks, analysed the four remaining files and reported no issues on any of them. The repository carries **audit/mythril.log** for the campaign as it ran and **audit/mythril-corrected-2026-08-29_1609.log** for the repaired run, with **audit/NOTE_MYTHRIL.md** explaining which is which. The failure is not hidden and the result is not missing.

Mythril printing "no issues detected" is not a proof of soundness. It prints the same line whether it explored the state space exhaustively or stopped on its execution timeout. On a contract this size, bounded symbolic execution does not cover everything. It means "nothing found within the budget", and it is consistent with Slither, Wake and Aderyn finding nothing new either.

The checklist, and the three findings a scan will report

SCSVS/SCSTG checklist: **227 items reviewed, no Critical, High or Medium item left open.** Separately, automated scanning raises two Critical and one High. All three are analysed as false positives, and the reasoning is written into the source rather than kept private. They are listed here so the numbers match what an independent check returns:

- The liquidity custodian permissionless **collect()** — the fees it moves can only reach the configured recipient, never a caller-chosen address.
- The airdrop unlock threshold — the scanner flags it as soon as the value drops below 10. Compiling the contract at 10 and at 5 and comparing the runtime bytecode shows the same length and two changed constant bytes, with no access-control instruction touched.
- A read-only reentrancy on the price read — closed by **require(slot0().unlocked)** and covered by a dedicated test.

13 What has not been done

NO THIRD-PARTY AUDIT OF \$RMHT HAS BEEN COMPLETED

This needs to be said plainly, because the opposite is easy to imply. **No independent firm has audited the \$RMHT contracts.** The SolidProof verification that the project carries is a **KYC check on the founder identity**, not a code audit. The SolidProof audit report that does exist covers **\$MHT V2 on BNB Chain** — a different contract, on a different chain, with a different architecture.

Everything in section 12 is work done by the author of the code, with tools anyone can run. It is more than many launches ever publish, and it is still not an external review. Automated tooling finds known patterns; it does not reason about whether the design does what it claims. That second question has not been answered by anyone independent, and until it is, this document will keep saying so.

Check it yourself

Every constant quoted in this document is a public variable, readable from any explorer without a wallet. The most useful ones: **vaultBalance** (what is left in the vault), **milestonesReached** and **nextMilestoneUSD** (how many milestones have fired and the next target), **milestoneStatus()** (armed or not, confirmations gathered, measured market cap), **getMarketCap()** (market cap as the contract computes it), **isTwapReady()** (whether the pool already covers the 30-

minute window), **owner()** and **feedGovernor()** (the governance state), and **pendingRewardsOf(address)** (what you can claim right now).

And every number in section 12 can be reproduced: clone github.com/dev-mht/rmht-contracts, run **forge test**, and compare with the logs archived in **audit/**. If a figure in this document does not match what the tools return, that is a defect in this document and it should be said publicly.

14 Links and risk notice

Resource	Address
Website	https://rmht.apexpad.io
Technical reference page	https://rmht.apexpad.io/protocol-en.html
Scanner findings answered	https://rmht.apexpad.io/transparency-en.html
Contracts (source)	https://github.com/dev-mht/rmht-contracts
Website repository	https://github.com/dev-mht/rmht
Explorer	https://robinhoodchain.blockscout.com
Market data	https://www.geckoterminal.com/robinhood/pools/0xabe3b1ff5fc6a9638d0c46f2379b20f7e6173bee
Telegram	https://t.me/Community_Milestone_hodl
X	https://x.com/milestone_hodl
Contact	milestonehodl.contact@proton.me
\$MHT on BNB Chain	https://milestonehodl.apexpad.io

RISK NOTICE

This document describes a technical mechanism. It is neither investment advice nor a promise of returns, and nothing in it should be read as a price expectation.

Digital assets are highly volatile and \$RMHT may lose all of its value. The contracts are unaudited by any third party. Liquidity that cannot be withdrawn is not liquidity that supports a price: the ETH in the pool is what buyers put there, and nothing guarantees a floor. Only commit what you can afford to lose entirely.

\$RMHT is not affiliated with, endorsed by or connected to Robinhood Markets, Inc. in any way. The name of the chain it runs on is not an endorsement.

This edition replaces the edition of 31 July 2026, which described an architecture that no longer exists. Where this document and the deployed code disagree, the code is authoritative.